Migration Report:

Where we started

In Assignment 01, all the SQL code for saving and reading data lived inside DatabaseManager. The services and the CSV import/export module called those methods directly. This meant the code that talks to the database and the code that handles the app's actual work were tangled together.

Where we want to end up

DatabaseManager now has one job: it opens the SQLite connection and sets up the tables. Two new classes, TransactionRepository and BudgetRepository, take over the SQL queries and turn database rows into the app's own objects.

The flow looks like this:

Services -> Repositories -> DatabaseManager -> SQLite database

We are not removing the old DatabaseManager methods yet. They stay as thin wrappers that pass the work on to the repositories. That way, anything still using the Assignment 01 code keeps working while we move the services over one at a time.

How to do the migration

1. Keep your existing SQLite file and run the app once. Your tables are left as they are, because the setup uses CREATE TABLE IF NOT EXISTS.
2. Add a new file: expense_mate/repositories.py.
3. Create a TransactionRepository and a BudgetRepository, giving each the existing DatabaseManager.
4. Check your saved transactions using get_all() and get_by_month().
5. Check your saved budgets using get() and get_for_month().
6. In the next round of refactoring, change the services so they use the repositories instead of DatabaseManager.

Nothing here deletes or copies any data. Your records stay in the same transactions and budgets tables.

Things to watch out for

Services that still call the old methods will keep working, because the wrappers pass their requests along.

The new repository methods use parameterized SQL, so the protection against bad input stays the same as before.

The month lookup uses date LIKE 'YYYY-MM-%'. This only works if dates are stored in the standard year-month-day format, but Transaction.validate() already makes sure they are.

How to check it worked

Run this command:

python -m unittest test_repositories.py test_expense_mate.py

The tests in test_repositories.py confirm that existing transactions are kept and that saving a budget twice updates it instead of duplicating it. The tests in test_expense_mate.py confirm that everything from Assignment 01 still works through the wrappers.